

Стратегия поставки ценности: от PoC до Production

Архитектурный подход к снижению
рисков и созданию успешных AI-проектов



Для студентов курса «AI Architect» и инженеров, стремящихся мыслить как технологические лидеры.



Проверить, идет ли запись

Меня хорошо видно & слышно?





Иван Четвериков

AI Architect

- Более 5 лет опыта в разработке и архитектуре
- Специалист в разработке и проектировании AI-приложений в Raft

 @istrebitel_1

 istrebitel.3.12@gmail.com



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ

Стратегический
фундамент и
планирование проекта

Проектирование и
документирование
архитектуры

Продвинутые
архитектурные
паттерны

Инфраструктура

Качество, интеграции и
безопасность

Стратегия, лидерство и
экономика

Свой проект

ФИНИШ



Почему большинство AI-проектов умирают, не дойдя до Production?

Большие проекты ломаются не из-за технологий, а из-за отсутствия стратегии.



Преждевременная сложность

Попытка сразу «сделать production», игнорируя промежуточные этапы.



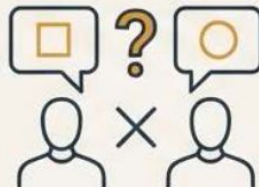
Ранняя оптимизация архитектуры

Инвестиции в масштабируемость и отказоустойчивость до того, как доказана ценность продукта.



Отсутствие бизнес-гипотезы

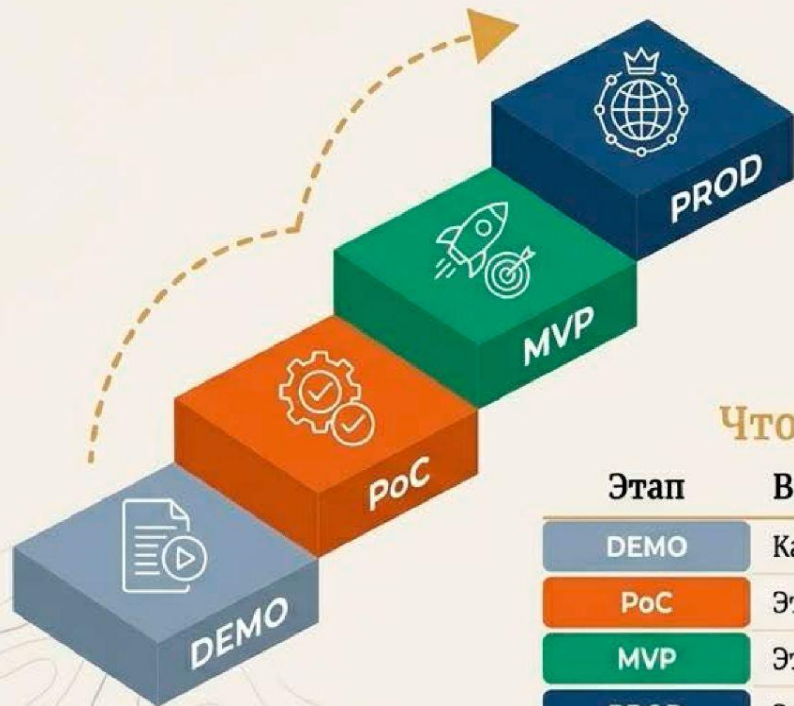
Разработка ведется без четкого понимания, какую проблему пользователя или бизнеса мы решаем.



Путаница в терминах

Команда и стейкхолдеры не понимают разницы между Demo, PoC, MVP и Production, что ведет к неверным ожиданиям.

Стратегический подъем: Путь от неопределенности к измеримой ценности



Мы рассматриваем разработку не как линейный процесс, а как поэтапное восхождение. Каждый этап — это стратегический шаг, который:

- Снижает неопределенность и отвечает на конкретный вопрос.
- Стоит дешевле следующего, минимизируя финансовые риски.
- Дает формальную точку остановки, если гипотеза не подтвердилась. Это не провал, а экономия ресурсов.

Что мы доказываем на каждом этапе?

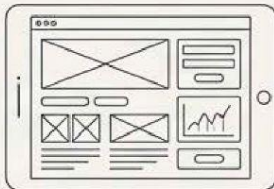
Этап	Вопрос	Аудитория
DEMO	Как это будет выглядеть?	Стейкхолдеры.
PoC	Это вообще технически возможно?	Инженеры, Техлиды.
MVP	Это кому-то действительно нужно?	Реальные пользователи.
PROD	Это стабильно и масштабируемо?	Бизнес, Операционная команда.

Этап 1: Demo. Проверяем, что все видят одну и ту же картину

«Мы правильно поняли задачу?»

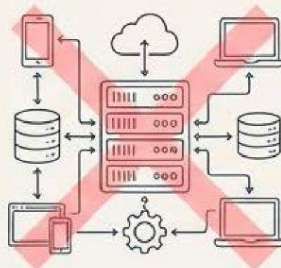
✓ ДЕМО — ЭТО:

- Визуализация идеи (UX-макет, кликабельный прототип).
- Презентация сценария использования.
- Использование фейковых данных и заглушек (Fake data / Stub API).
- **Цель:** Выровнять ожидания и получить быструю обратную связь.



✗ ДЕМО — ЭТО НЕ:

- Рабочая архитектура.
- Production-ready код.
- Масштабируемое решение.
- Основа для будущей разработки.



[Architect's Insight Callout]: "Самый дешевый способ услышать «нет, мы хотели совсем другое»."

Этап 2: PoC (Proof of Concept). Отвечаем на вопрос: «Это в принципе возможно?»

Основная цель: Проверка ключевой технической гипотезы. Мы изолируем самую сложную и рискованную часть системы и доказываем ее жизнеспособность.

Что мы проверяем



Технологии: Подходит ли выбранный стек?



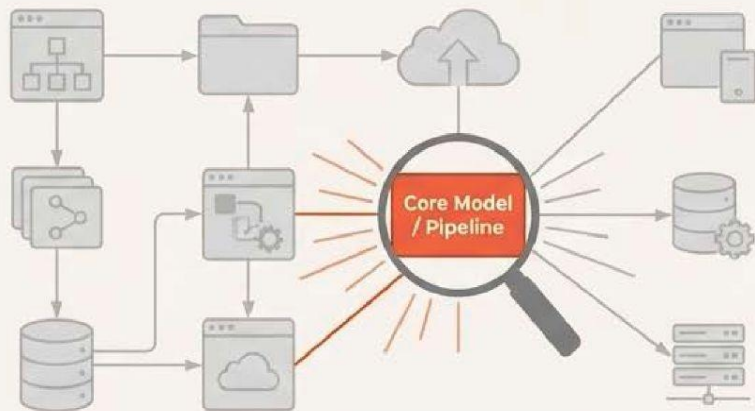
Данные: Достаточно ли данных, какого они качества?



Алгоритмы: Дает ли модель приемлемое качество?



Производительность: Укладываемся ли в ограничения (latency, throughput)?



Типовые PoC-активности:

- Разведочный анализ данных (EDA).
- Тестирование core-модели или ключевого пайплайна.
- Spike-решения для проверки интеграций.
- Измерение базовых технических метрик.
- «Data Challenge» для оценки выполнимости.

Архитектура PoC: Осознанные компромиссы

Ключевой принцип: Скорость исследования важнее качества кода и инфраструктуры.

Типичная архитектура PoC

Jupyter-ноутбуки /
Набор скриптов

Монолит

Локальное окружение
или один dev-сервер

Минимум или
полное
отсутствие
автоматизации

На что мы сознательно идем



Нет отказоустойчивости:
Если упадет,
перезапустим руками.



Нет CI/CD:
Деплой через ``scp``
или ``git pull``.



Нет безопасности:
Security hardening
отсутствует.



Нет observability:
Логи смотрим в
консоли.

Результат PoC



Вывод: Однозначный ответ GO / NO-GO на дальнейшую разработку.

Артефакты: Список остаточных технических рисков и более точная оценка сложности MVP.

Этап 3: MVP (Minimum Viable Product). Проверяем главную гипотезу: «Готов ли кто-то этим пользоваться?»

MVP ≠ «маленький production»: Это инструмент для проверки рыночной гипотезы с минимальными затратами.

Ключевые характеристики MVP

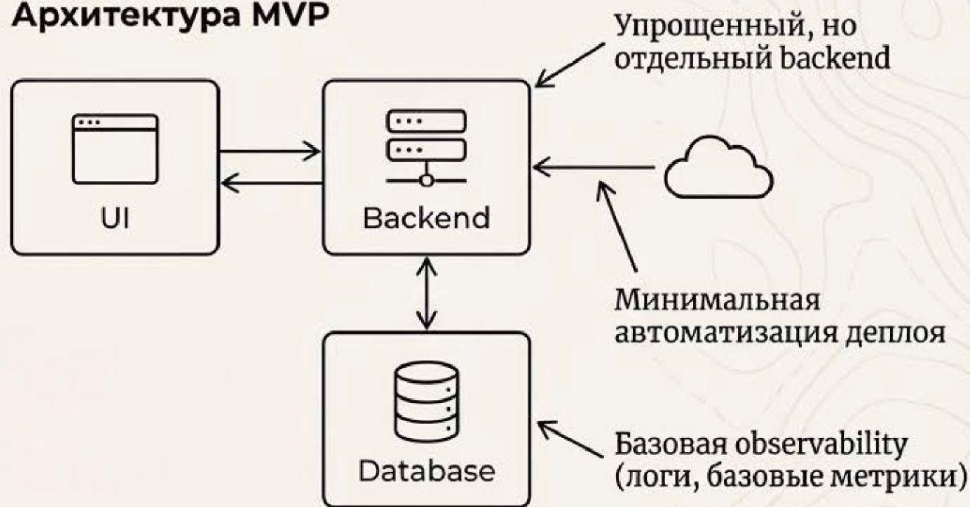
 **Реальные пользователи:** Не команда, не стейкхолдеры.

 **Реальные данные и сценарии:** Решает одну, но настоящую проблему.

 **Минимальный, но достаточный функционал:** Фокус на одной ключевой use-case.

 **Ценность:** Пользователь уже должен получать измеримую пользу.

Архитектура MVP



[Architect's Insight Callout] Архитектура MVP должна позволять его выбросить. Если гипотеза не подтвердилась, это не технический долг, а закрытые инвестиции в знание.

Метрики .!MVP: Как понять, что мы движемся в правильном направлении?

Успех MVP измеряется не техническими показателями, а изменением поведения пользователей.

Пирамида метрик MVP



Инженерная стратегия и контракт: Как этапы разработки защищают бюджет и команду

Выбор этапа напрямую влияет на тип контракта и управление финансовыми рисками.

Demo / PoC



Time & Material (T&M)

Высокая неопределенность. Заказчик платит за время и экспертизу команды для исследования. Риски на стороне заказчика.

MVP



Гибридный (T&M с ограничением или поэтапный Fixed Price)

Неопределенность снижена, но гипотеза еще не доказана. Бюджет фиксируется на короткие итерации.

Production



Fixed Price / SLA

Требования ясны, технология доказана. Подрядчик может взять на себя риски по срокам и бюджету в обмен на фиксированную оплату.

Почему это важно инженеру-архитектору?

- Это напрямую определяет ожидания от вашей работы.
- Это формирует бюджет на инфраструктуру и команду.
- Это защищает команду от давления «сделать всё и сразу» в условиях неопределенности.

Этап 4: **Production.** Это не про код, это про надежность и процессы

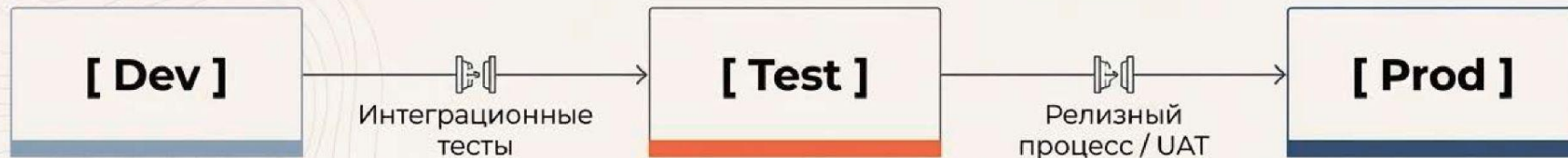
Код, который работает на ноутбуке — это артефакт.
Система, которой доверяет бизнес — это **Production**.

Компоненты Production-системы



Архитектура окружений: Баланс между скоростью разработки и стабильностью Production

Среда	Главная цель	Ключевые особенности
<u>Dev</u>	Скорость и гибкость	Максимальная свобода для разработчиков. Нестабильность — это норма. Быстрые и частые изменения.
<u>Test/Staging</u>	Стабильность и контроль	Идентичное или близкое к Production окружение. Интеграционные тесты, UAT. Контролируемые релизные циклы.
<u>Prod</u>	Надежность и SLA	Максимальная стабильность. Изменения вносятся только по строгим процедурам. Жесткий контроль доступа. Выполнение бизнес-обязательств.



Архитектурная эволюция: Почему PoC нельзя просто скопировать в Production

Переход от PoC → MVP → Prod — это не копирование кода, а осознанная переработка архитектуры на каждом этапе.



Ключевые принципы эволюции

- **Осознанная переработка:** На каждом этапе мы пересматриваем компромиссы предыдущего и принимаем новые решения.
- **Стратегический техдолг:** В PoC и MVP мы сознательно закладываем техдолг (например, в отсутствии тестов), чтобы ускорить проверку гипотез. На пути в Production мы должны его планомерно погасить.
- **Инкрементальные улучшения:** Не пытайтесь переписать все с нуля. Улучшайте систему шаг за шагом.

Карта стратегического подъема: от идеи до ценности





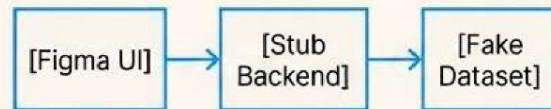
Теория ясна. Посмотрим, как это работает на практике.

Практический кейс: Разработка дорожной карты для AI-системы рекомендаций в ритейле

Контекст проекта:

- **Компания:** Средний ритейлер (50–100 магазинов).
- **Проблема:** Избыточные остатки и out-of-stock, решения принимаются вручную.
- **Бизнес-гипотеза:** Использование ML-модели прогнозирования спроса и рекомендаций по заказам позволит снизить списания на 10–15% и повысить availability на полке.

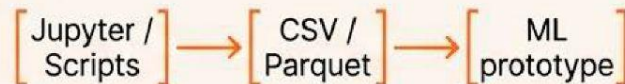
■ Кейс. Этап 1: Демо — Фасад будущего решения



Синхронизировать ожидания бизнеса и команды, проверить, что предлагаемое решение отвечает реальной боли.

 Score (Что делаем)	 Риски	 Критерии успеха
• Clickable-прототип интерфейса • Fake data (Excel / CSV) • Stub API (заглушки) НЕ делаем: реальный ML, интеграции.	• Неверное понимание процесса закупок • Лишние или недостающие поля в интерфейсе • Ожидание «чуда от AI»	Бизнес говорит:  «Да, это то, чем мы будем пользоваться»

■ Кейс. Этап 2: PoC — Проверка технической гипотезы

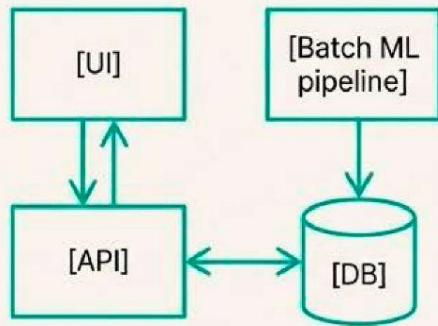


Цель: Понять, даёт ли ML хоть какой-то сигнал, и оценить сложность следующего шага.

 Score (Что делаем)	 Риски	 Критерии успеха
• Загрузка исторических продаж • EDA (сезонность, промо) • Прототип модели (baseline + LightGBM) • Прототип модели (baseline + LightGBM) НЕ делаем: real-time, CI/CD, безопасность.	• Данные слишком шумные • Данные слишком шумные • Нет стабильной корреляции • Требуются внешние факторы	• ML-модель лучше baseline минимум на X% • Понятен список baseline минимум на X% • Понятен список data gaps и рисков для MVP Решение: 🖱️ Go / No-Go

Кейс. Этап 3: MVP — Проверка рыночной гипотезы

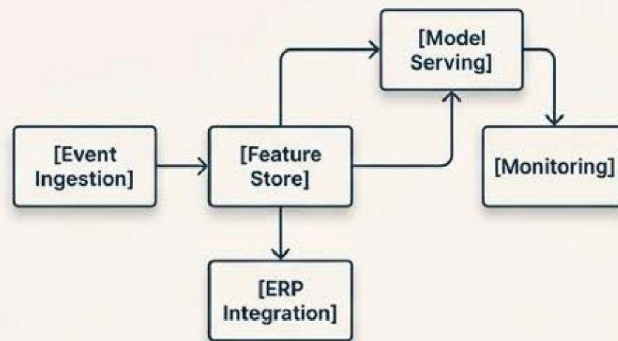
Цель: Проверить ценность для пользователей и получить первые измеримые бизнес-эффекты.



 Scope (Что делаем)	 Риски	 Метрики успеха
• Один регион / 5 магазинов • Один сценарий: рекомендация заказа • Реальные данные (batch) • Логирование решений НЕ делаем: масштабирование, сложные модели.	• Пользователи не доверяют рекомендациям • Изменение процессов важнее точности модели • Модель «хорошая», но неудобная	Бизнес: % принятых рекомендаций, ↓ списаний Продукт: Retention, Usage frequency Техн.: Latency batch < X, стабильность пайплайна

■ Кейс. Этап 4: Production — Промышленная система

Цель: Встроить решение в core-процессы, масштабировать на всю сеть, обеспечить предсказуемые SLA.



 Scope (Что делаем)	 Риски	 Критерии успеха
• Full data pipeline • CI/CD • Monitoring & alerts • Security • Retraining • Интеграция с ERP	• Рост стоимости поддержки • Data drift • Организационные изменения	• SLA соблюдаются • Модель используется в >X% случаев • Экономический эффект подтверждён

Главные выводы: Мышление архитектора



Ключевая трансформация мышления

- Не «рисовать финальную архитектуру сразу».
- Мыслить шагами, последовательно снижая риски.
- Говорить с бизнесом на одном языке (цели, риски, критерии успеха).

Вы научились

- Объяснять **зачем** нужен каждый этап.
- Защищать бюджет для каждого шага.
- Сказать **«стоп»** вовремя, экономя ресурсы.

Вопросы



если есть вопросы



если вопросов нет

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

